

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studentii din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată gestionării unei linii inteligente de producție dintr-o fabrică.

3p. Ești dezvoltator software la o companie care produce echipamente personalizate pe o linie automată de asamblare. Aplicația trebuie să gestioneze atât accesul controlat la linia de producție, cât și generarea rapidă a unor configurații repetitive de produse. Un angajat poate iniția o sesiune de lucru asupra liniei de producție, iar din acel moment doar acel angajat are dreptul să efectueze modificări până când finalizează intervenția. Se pot realiza operații precum: pornirea liniei, oprirea liniei, schimbarea vitezei de lucru, schimbarea modului de operare sau adăugarea unei etape tehnologice. Dacă un alt angajat încearcă să modifice linia cât timp aceasta este rezervată de primul angajat, sistemul trebuie să arunce o excepție. De asemenea, eliberarea liniei trebuie să poată fi făcută doar de angajatul care a inițiat sesiunea curentă.

1.5p. Să se testeze soluția prin inițierea unei sesiuni de lucru de către un angajat, realizarea a cel puțin 2 modificări valide asupra liniei de către același angajat și încercarea unui al doilea angajat de a modifica linia, situație în care trebuie demonstrată apariția excepției.

3p. Fabrica produce frecvent loturi de produse foarte asemănătoare, iar definirea completă a configurației fiecărui produs de la zero consumă timp. Pentru optimizarea procesului, aplicația trebuie să permită reutilizarea unora din cele 2-3 configurații standard stocate cu rol de reutilizare facilă. Un produs configurabil este de tip *ProductionItem* iar deoarece multe comenzi pornesc de la aceleași șabloane validate anterior, se cere implementarea unui mecanism

prin care un obiect de tip produs să poată fi duplicat rapid, iar copia să poată fi ulterior personalizată.

1.5p. Să se testeze soluția prin:

- definirea a cel puțin 2 prototipuri diferite de produse;
- clonarea unui prototip de cel puțin 2 ori;
- particularizarea fiecărei clone și demonstrarea faptului că prototipul inițial rămâne nemodificat.

```
public interface IProductionLineControl {  
    void startLine(String employeeId) throws Exception;  
    void stopLine(String employeeId) throws Exception;  
    void setWorkingSpeed(String employeeId, int speed) throws Exception;  
    void setOperatingMode(String employeeId, String mode) throws Exception;  
    void addTechnologicalStep(String employeeId, String step) throws Exception;  
}
```

```
public interface ProductionItem {  
    String getModelName();  
    String getSerialCode();  
    String getBatchLabel();  
    String getPackagingType();  
    void displayInfo();  
}
```